

# mU-Code Interpreter

가상 머신 설계 및 구현 계획 발표

팀 UCI

컴퓨터공학과 20220744 이강석(팀장)  
컴퓨터공학과 20240058 권기영



PROBLEM ANALYSIS & DESIGN

# 문제 분석 및 설계 개요

## 01 문제 분석 및 설계 개요

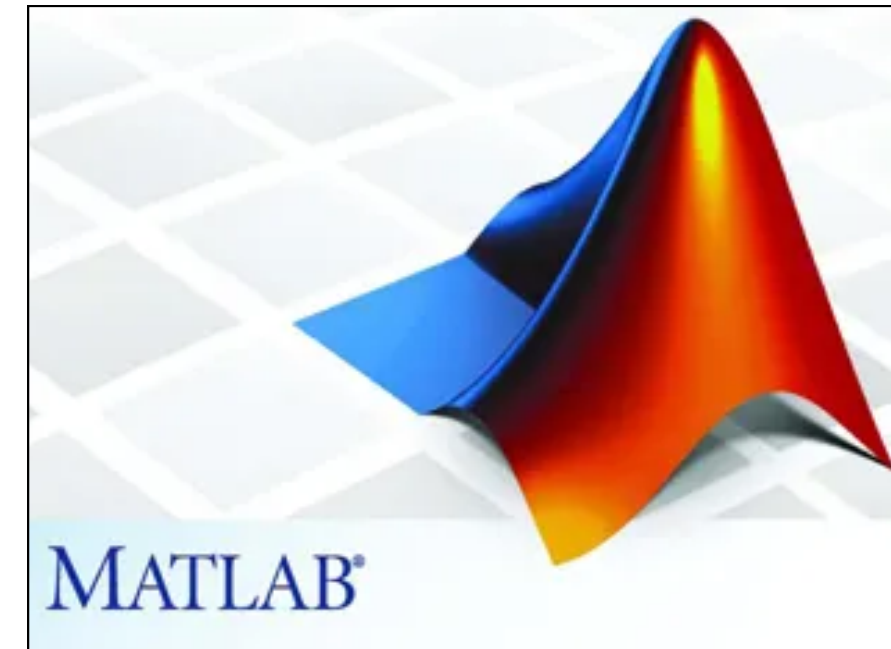
### 소프트웨어 프로젝트 3단계 - mU-Code Interpreter

#### 프로젝트 목표

mU-Code를 실행할 수 있는 인터프리터 구현

#### 프로젝트 범위

- mU-Code를 실행할 수 있는 단일 가상 머신 구조 구현
- 어셈블러를 통한 mU-Code 변환 및 로드
- 인터프리터의 명령어 인출, 분석, 실행 과정 구현
- mU-Code 명령어 처리 및 실행 통계 분석 후 lst 파일 생성
- 프로그램 실행 과정을 시각적으로 확인 가능한 GUI 구현



대표적인 인터프리터(MATLAB, python)

## 01 문제 분석 및 설계 개요

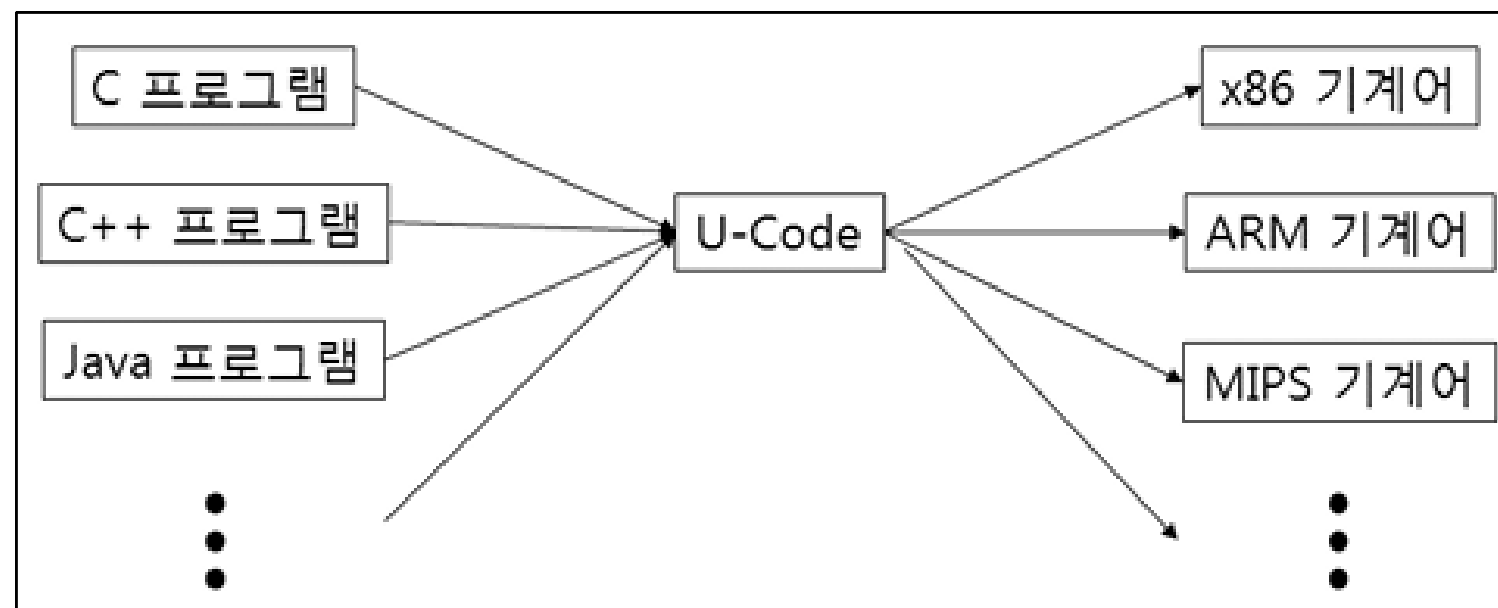
### mU-Code

#### mU-Code란?

스택 기반으로 동작하는 간단한 CPU에서 실행되는 것을 가정한 중간 언어로, Pascal 언어의 P-Code를 일반화한 것

#### 중간 언어란?

고급 언어를 바로 기계어로 바꾸지 않고, 가상 머신의 최적화나 이식성을 위해 중간에 바꿔놓은 언어



중간 언어의 역할

#### mU-Code 특징

- 모든 데이터는 정수형이며, 데이터 크기는 바이트가 아닌 개수로 표현함
- CPU 내 레지스터가 없고, 모든 연산은 스택 기반(0-addressing)으로 수행됨
- 상수, 메모리 접근, 분기 명령어는 1개의 피연산자(1-addressing)를 사용함
- 코드 작성 시 고정된 열 형식을 따름: 1~10열은 레이블, 11열은 공백, 12열 이후 명령어 기입
- 명령어와 피연산자 사이에는 공백이 반드시 필요함
- '%'로 시작하는 구문은 주석으로 처리되어 실행되지 않음



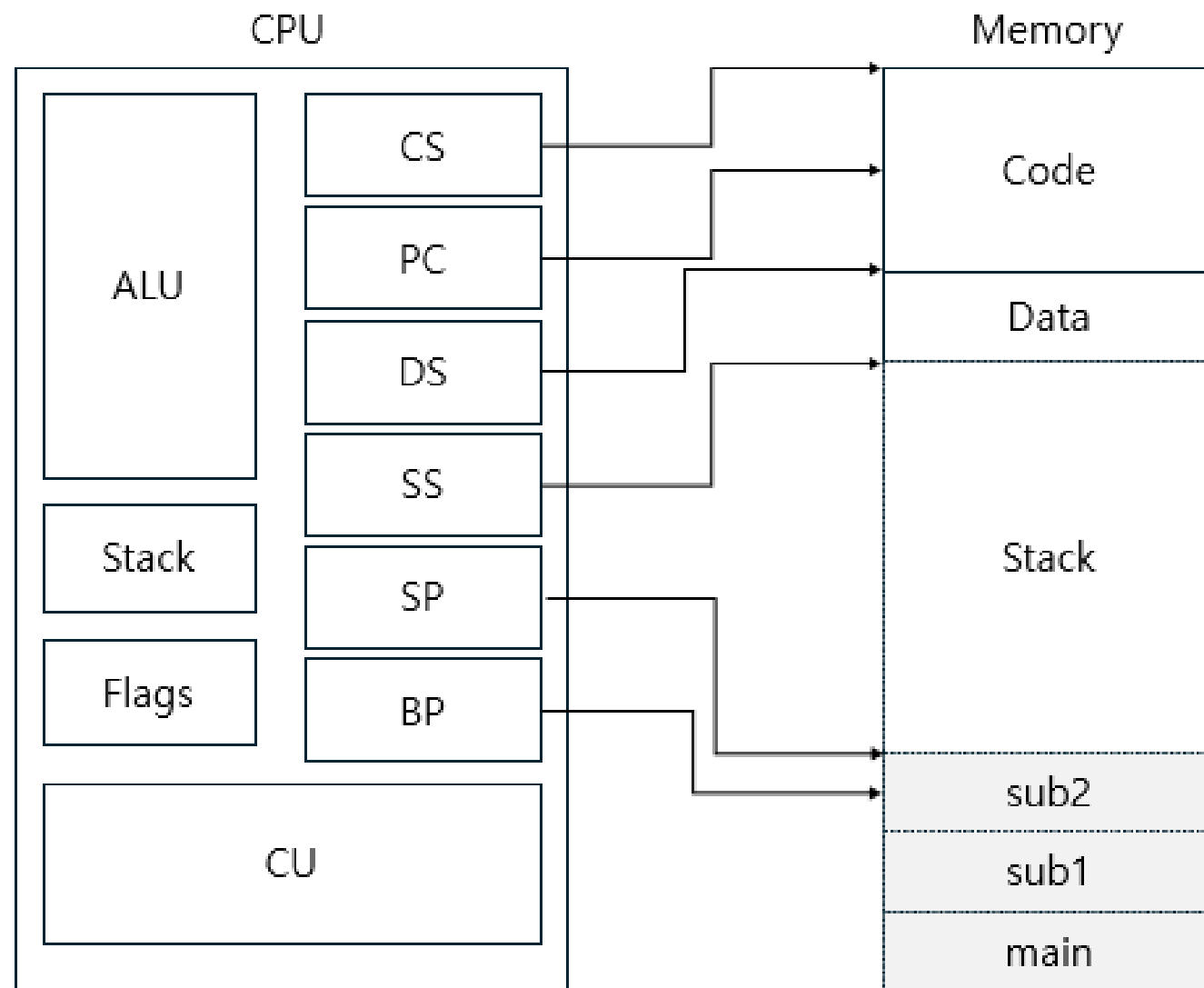
01 문제 분석 및 설계 개요

mU-Code 명령어

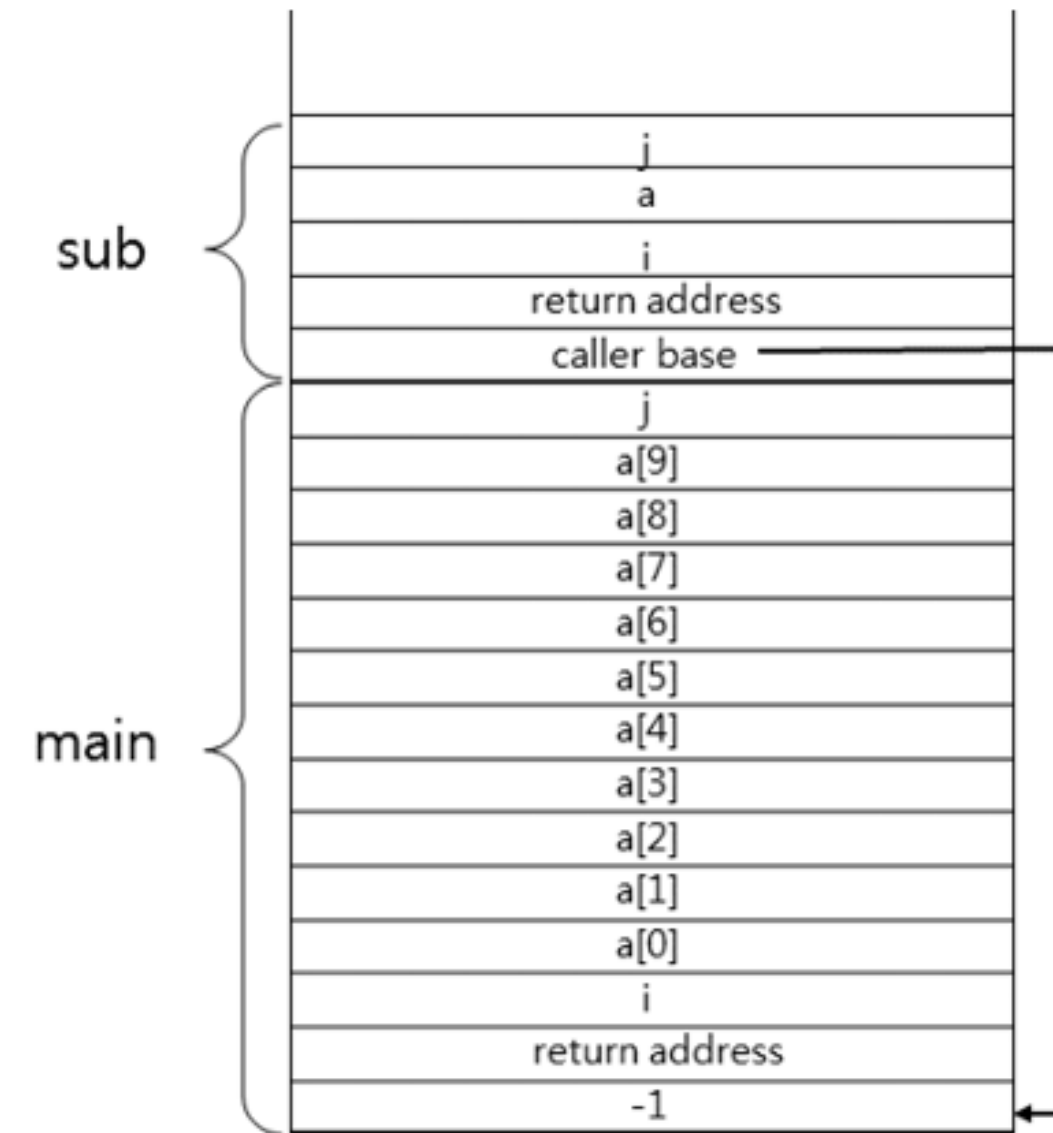
| 명령어 종류     | 명령어                                                       | 비고                             |
|------------|-----------------------------------------------------------|--------------------------------|
| 프로그램 구성 명령 | nop, bgn n, sym b n s, end                                | 어셈블 단계에서만 활용                   |
| 입출력 처리     | read(i), write(i), lf()                                   | 시스템 함수 사용 예)call read          |
| 함수 정의 및 호출 | proc n, ret, ldp, push, call label                        |                                |
| 흐름 제어      | ujp labelm, tjp label, fjp label                          |                                |
| 데이터 이동 연산자 | lod b n, lda b n, ldc c, str b n, ldi, sti                | 메모리에서 CPU 스택, CPU 스택에서 메모리로 이동 |
| 이항 연산자     | add, sub, mult, div, mod, gt, lt, ge, le ,eq, ne, and, or | CPU 스택 활용                      |
| 단항 연산자     | not, neg                                                  | CPU 스택 활용                      |

## 01 문제 분석 및 설계 개요

### 가상 머신 구조



- 기본적으로 ALU, CU, Stack, Memory의 구조 사용
- 하나의 메모리를 프로그램 코드, 전역변수, 지역변수 영역으로 나눠 사용
- 메모리 구분 및 관리에는 특수 레지스터를 사용함
- 에러 처리를 위한 플래그



- 함수 종료시 복귀할 주소와 BP 레지스터의 값을 메모리 스택에 저장



## 01 문제 분석 및 설계 개요

## mU-Code 예시(.uco)

| mU-Code  |                            | 해설                              | CPU 스택 |                      |
|----------|----------------------------|---------------------------------|--------|----------------------|
| main     | bgn 1                      | 프로그램의 시작 - 전역 변수의 개수는 1         | 10     |                      |
|          | sym 1 0 1                  | MAX 변수의 블록(1)과 오프셋(0)과 크기(1) 설정 |        |                      |
|          | ldc 10                     | 상수 10을 스택에 적재                   |        |                      |
|          | str 1 0                    | 스택의 값을 블록 1, 오프셋 0에 저장          |        |                      |
|          | ldp                        | 함수 호출 준비 명령                     |        |                      |
|          | call main                  | main 함수 호출(항상 main을 호출해야 함)     |        |                      |
|          | end                        | 프로그램 종료                         |        |                      |
|          | proc 12                    | 함수 시작 - 지역 변수의 총량 선언            |        |                      |
|          | sym 2 0 1                  | i 변수의 블록(2)과 오프셋(0)과 크기(1) 설정   |        |                      |
|          | sym 2 1 10                 | a 배열의 블록(2)과 오프셋(1)과 크기(10) 설정  |        |                      |
|          | sym 2 11 1                 | j 변수의 블록(2)과 오프셋(11)과 크기(1) 설정  | 0      |                      |
|          | ldc 0                      | 상수 0을 스택에 적재                    |        |                      |
|          | dup                        | 스택의 값을 복사하여 스택에 적재              |        | 0 0                  |
|          | str 2 11                   | 스택의 값을 블록 1, 오프셋 11에 저장 (j = 0) |        | 0                    |
|          | str 2 0                    | 스택의 값을 블록 1, 오프셋 0에 저장 (i = 0)  |        |                      |
| \$0      | nop                        | while 문의 시작점 - 반복 시 돌아올 위치 \$0  | 0      |                      |
|          | lod 2 0                    | 블록 2, 오프셋 0의 값을 스택에 적재(i)       |        |                      |
|          | lod 1 0                    | 블록 1, 오프셋 0의 값을 스택에 적재(MAX)     |        | 0 10                 |
|          | lt                         | 스택 꼭대기의 두 값을 비교하여 결과를 적재        |        | -1 <sup>(1)</sup>    |
|          | fjp \$1                    | 스택의 값이 거짓이면 \$1로 점프(반복 종료)      |        |                      |
|          | lod 2 0                    | 블록 2, 오프셋 0의 값을 스택에 적재(i)       |        | 0                    |
|          | lda 2 1                    | 블록 2, 오프셋 1의 주소를 적재(a의 주소)      |        | 0 2 <sup>(2)</sup>   |
|          | add                        | 스택 꼭대기의 두 값을 더하여 결과를 적재(a[i])   |        | 2                    |
|          | lod 2 11                   | 블록 2, 오프셋 11의 값을 스택에 적재(j)      |        | 2 0                  |
|          | sti                        | 스택 top의 값을 top-1번지에 저장          |        |                      |
|          | lod 2 0                    | 블록 2, 오프셋 0의 값을 스택에 적재(i)       |        | 0                    |
|          | lda 2 1                    | 블록 2, 오프셋 1의 주소를 적재(a의 주소)      |        | 0 2                  |
|          | add                        | 스택 꼭대기의 두 값을 더하여 결과를 적재(a[i])   |        | 2                    |
|          | ldp                        | 함수 호출 준비                        |        | 2                    |
|          | lod 2 0                    | 블록 2, 오프셋 0의 값을 스택에 적재(i)       |        | 2 0                  |
|          | push                       | 스택의 값을 메모리 스택에 적재               |        | 2                    |
|          | lda 2 1                    | 블록 2, 오프셋 1의 주소를 적재(a의 주소)      |        | 2 2                  |
|          | push                       | 스택의 값을 메모리 스택에 적재               |        | 2                    |
|          | call sub                   | sub 함수 호출                       |        | 2 100 <sup>(3)</sup> |
|          | sti                        | 스택 top의 값을 top-1번지에 저장          |        | 0                    |
| lod 2 11 | 블록 2, 오프셋 11의 값을 스택에 적재(j) |                                 |        |                      |

| mU-Code       | 해설                                       | CPU 스택               |
|---------------|------------------------------------------|----------------------|
| lod 2 0       | 블록 2, 오프셋 0의 값을 스택에 적재(i)                | 0 0                  |
| lda 2 1       | 블록 2, 오프셋 1의 주소를 적재(a의 주소)               | 0 0 2                |
| add           | 스택 꼭대기의 두 값을 더하여 결과를 적재(a[i])            | 0 2                  |
| ldi           | 스택 값의 번지에 있는 값을 적재                       | 0 100                |
| add           | 스택 꼭대기의 두 값을 더하여 결과를 적재                  | 100                  |
| str 2 11      | 스택의 값을 블록 2, 오프셋 11에 저장(j)               |                      |
| lod 2 0       | 블록 2, 오프셋 0의 값을 스택에 적재(i)                | 0                    |
| ldc 1         | 상수 1을 스택에 적재                             | 0 1                  |
| add           | 스택 꼭대기의 두 값을 더하여 결과를 적재(i+1)             | 1                    |
| str 2 0       | 스택의 값을 블록 2, 오프셋 0에 저장(i)                |                      |
| ujp \$\$0     | \$\$0 위치로 무조건 점프                         |                      |
| \$ \$1<br>nop | \$ \$1 위치 표시 (아무 동작 없음 - nop)            |                      |
| ldp           | 함수 호출 전 인자 적재                            |                      |
| lod 2 11      | 블록 2, 오프셋 12의 값을 스택에 적재(j)               | 52405 <sup>(4)</sup> |
| push          | 스택의 값을 메모리 스택에 적재                        |                      |
| call write    | 값을 출력                                    |                      |
| ret           | main 함수 종료                               |                      |
| sub<br>proc 3 | 함수 시작 - 지역 변수의 총량 선언                     | 2                    |
| sym 3 0 1     | i 인자의 블록(3)과 오프셋(0)과 크기(1) 설정            | 2                    |
| sym 3 1 1     | a 인자의 블록(3)과 오프셋(1)과 크기(1) 설정            | 2                    |
| sym 3 2 1     | j 변수의 블록(3)과 오프셋(2)과 크기(1) 설정            | 2                    |
| ldp           | 함수 호출 전 인자 적재                            | 2                    |
| lda 3 2       | 블록 3, 오프셋 2의 주소를 스택에 적재(j 주소)            | 2 15 <sup>(5)</sup>  |
| push          | 스택의 값을 메모리 스택에 적재                        | 2                    |
| call read     | read 함수 호출. 지정 주소에 입력값 저장                | 2                    |
| lod 3 2       | 블록 3, 오프셋 2의 값을 스택에 적재(j)                | 2 100 <sup>(6)</sup> |
| lod 3 0       | 블록 3, 오프셋 0의 값을 스택에 적재(i)                | 2 100 0              |
| lod 3 1       | 블록 3, 오프셋 1의 값을 스택에 적재(a) <sup>(7)</sup> | 2 100 0 2            |
| add           | 스택 꼭대기의 두 값을 더하여 결과를 적재(a[i])            | 2 100 2              |
| ldi           | 스택 값의 번지에 있는 값을 적재                       | 2 100 0              |
| add           | 스택 꼭대기의 두 값을 더하여 결과를 적재                  | 2 100                |
| str 3 2       | 스택의 값을 블록 3, 오프셋 2에 저장(j)                | 2                    |
| lod 3 2       | 블록 3, 오프셋 2의 값을 스택에 적재                   | 2 100                |
| ret           | sub 함수 종료                                | 2 100                |



## 01 문제 분석 및 설계 개요

### 프로그램 실행 결과 양식(.lst)

어셈블 결과

원본 코드

실행 결과

명령어 사용 횟수  
명령어 실행 횟수

...

| addr | object | mUcode | source | program |
|------|--------|--------|--------|---------|
|------|--------|--------|--------|---------|

|    |        |      |      |    |
|----|--------|------|------|----|
| 0  | 37 0   |      | bgn  | 1  |
| 2  | 20 10  | ldc  | 10   |    |
| 4  | 24 1 0 | str  | 1 0  |    |
| 7  | 23     | ldp  |      |    |
| 8  | 29 11  | call | main |    |
| 10 | 36     | end  |      |    |
| 11 | 35 12  | main | proc | 12 |
| 13 | 20 0   | ldc  | 0    |    |

... (중간 생략)

=====실행 결과=====

52405

=====

=====명령어별 사용 횟수 (Instruction Use Count)=====

add = 5    lt = 1    lod = 12

... (중간 생략)

=====명령어별 실행 횟수 (Instruction Run Count)=====

add = 20    lt = 10    lod = 98

... (중간 생략)

=====기타=====

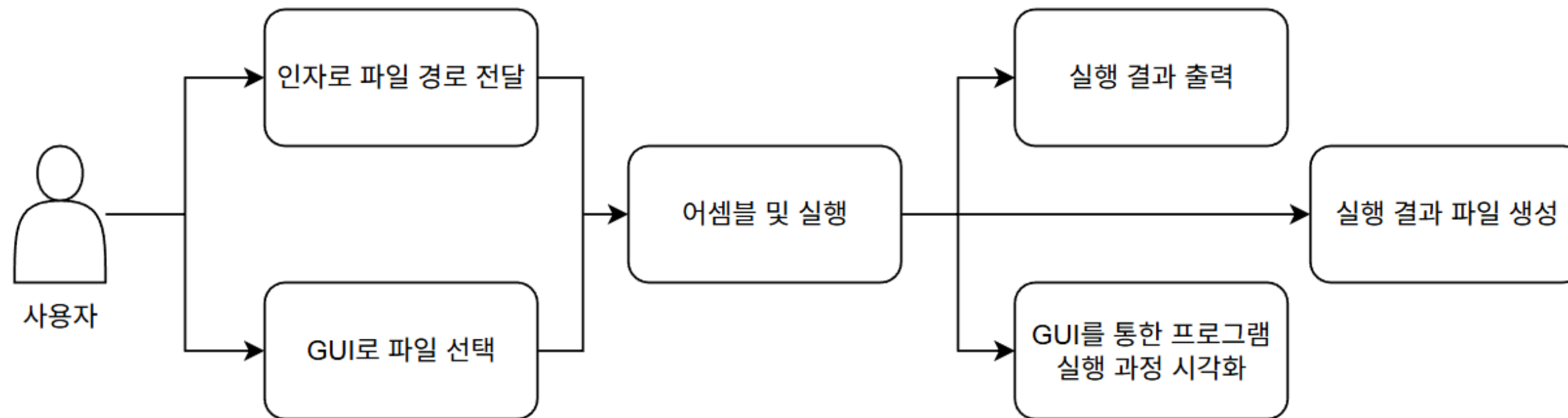
메모리 접근 횟수 = 155

SCENARIO & REQUIREMENTS

# 시나리오 및 요구사항

## 02 시나리오 및 요구사항 - 시나리오

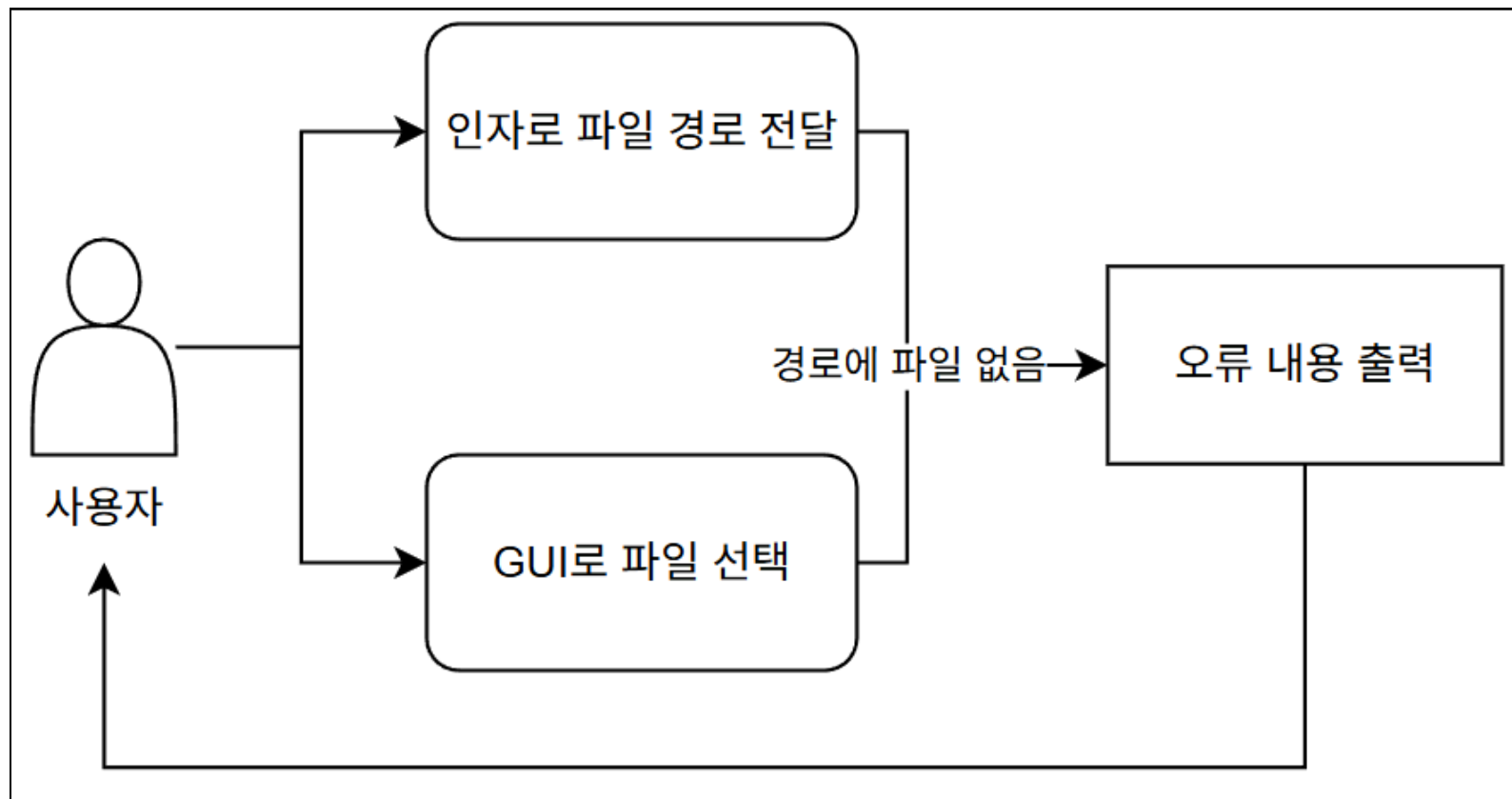
### 성공 시나리오



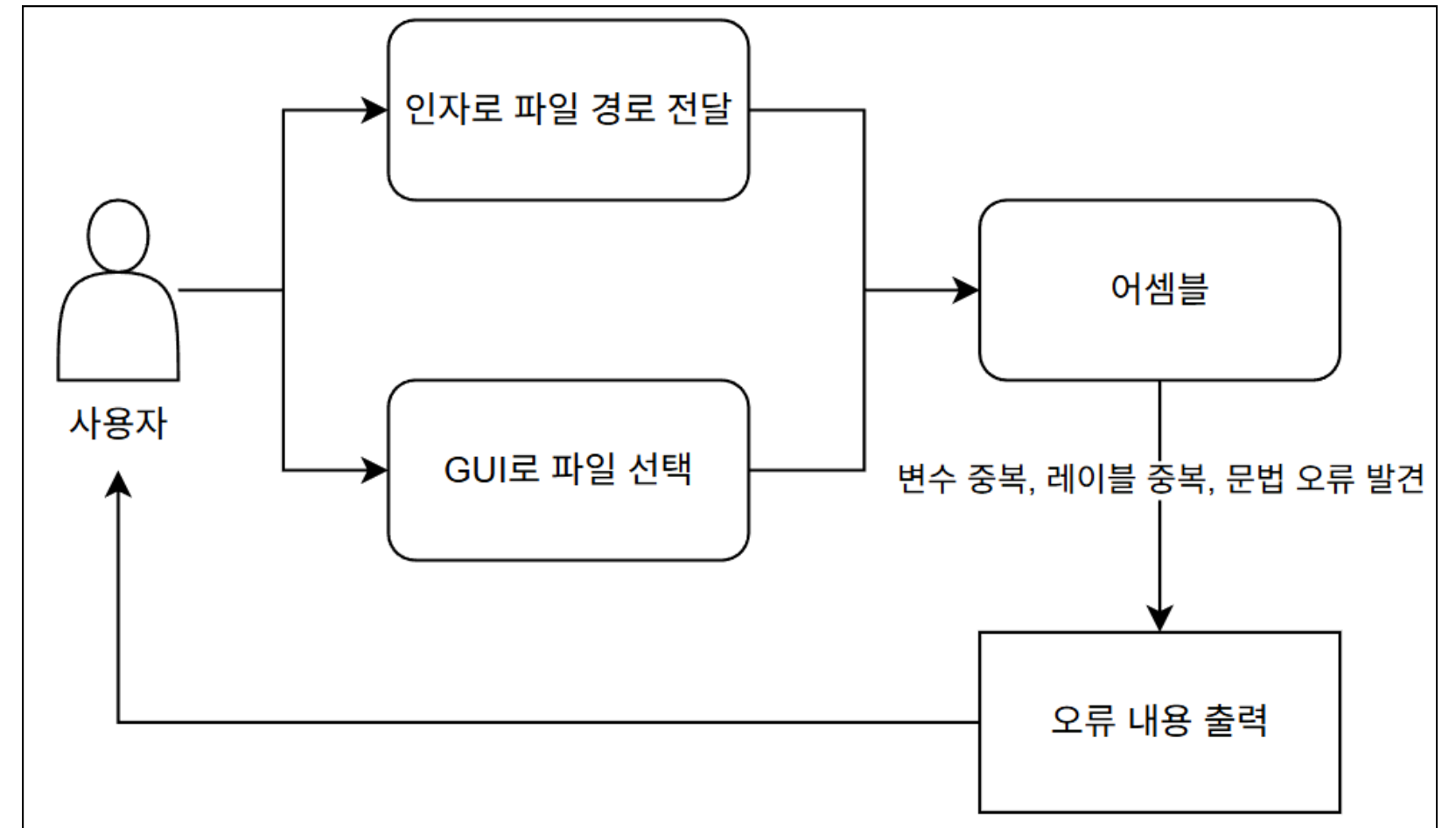
1. 사용자는 작성한 .uco 파일을 선택한 인터페이스를 통해 제공
2. 입력된 파일 어셈블 및 실행
3. 선택한 인터페이스를 통한 실행 결과 처리
4. 실행결과 .lst 파일 생성

## 02 시나리오 및 요구사항 - 시나리오

### 대체 시나리오1



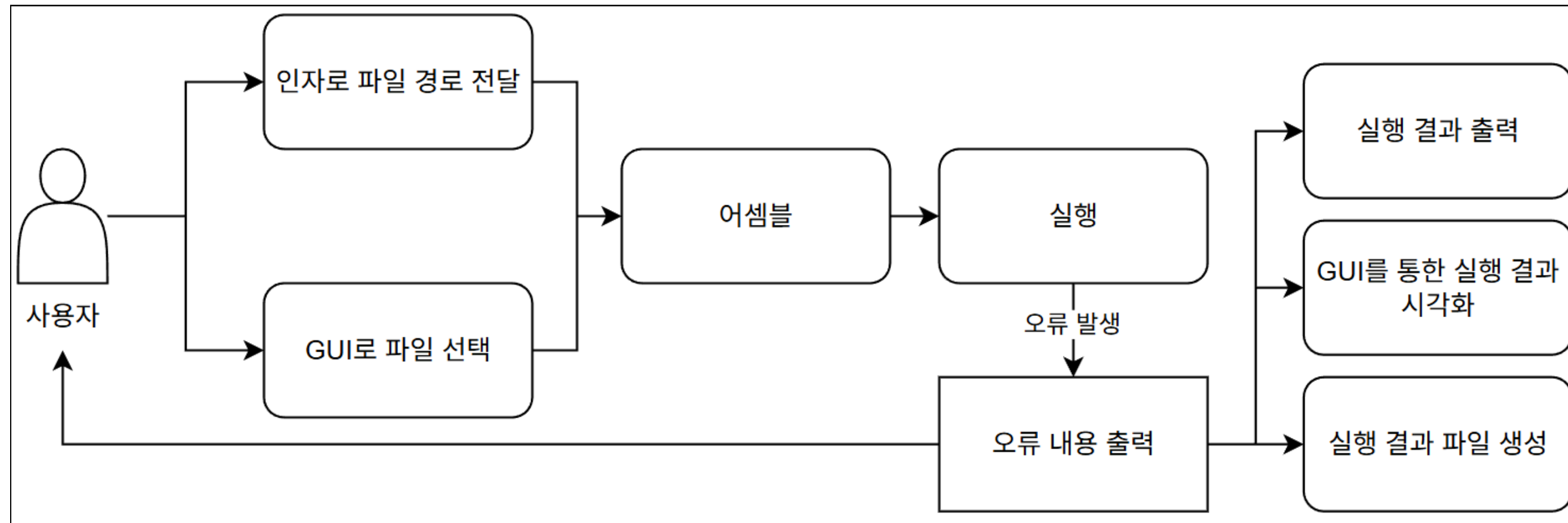
- 입력받은 경로에 파일이 존재하지 않는 경우
- 오류 내용 출력 후 종료



- 어셈블 단계에서 변수 중복, 레이블 중복, 문법 오류 발견한 경우
- 어느 부분에 오류가 있는지 출력 후 종료

## 02 시나리오 및 요구사항 - 시나리오

### 대체 시나리오2



- 실행 단계에서 스택 오버플로우, 잘못된 메모리 접근 등의 오류가 발생한 경우
- 실행 단계 종료 후 결과 처리



## 02 시나리오 및 요구사항 - 요구사항 분석

### 기능 요구사항 - 사용자 기능



#### .uco 파일 입력

CLI 인자 또는 GUI “파일 열기”로 선택 → 내부 로드  
(오류 시 재입력 요청)



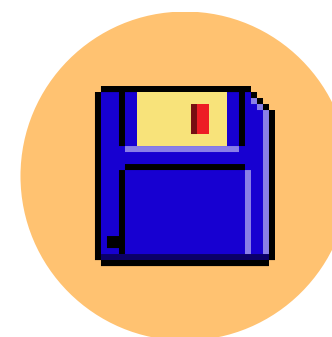
#### 코드 실행 및 결과 확인

실행 버튼으로 전체 코드 실행, 필요 시 입력 받음  
(오류 시 실행 중단 및 메시지 표시)



#### 단계별 실행 결과 확인

실행 완료 후 기록 조회, 특정 단계 선택 가능  
(로그 손상 시 경고 표시)



#### 통계 및 결과 저장

사용자 지정 경로로 .lst 파일 생성  
(저장 실패 시 오류 메시지)

## 02 시나리오 및 요구사항 - 요구사항 분석

### 기능 요구사항 - 시스템 기능



#### 어셈블 처리

.uco 파일 구문분석 → opcode/피연산자/레이블  
처리 → code 영역 구성  
(오류 시 라인 번호 및 로그 출력)



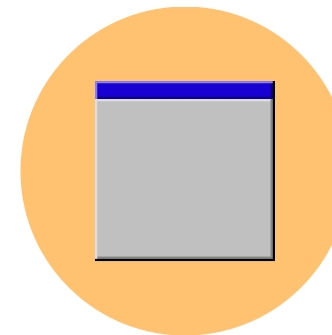
#### 실행 엔진

순차적 명령어 실행, 메모리·레지스터 관리  
(잘못된 메모리 접근 시 오류 출력 후 종료)



#### 오류/예외 처리

파일 오류, 구문 분석 실패, 실행 중 예외 등 감지  
→ GUI 경고



#### GUI 이벤트 처리

버튼 클릭, 파일 선택, 종료 등 신호 기반 처리



## 02 시나리오 및 요구사항 - 요구사항 분석

### 비기능 요구사항

#### 성능 및 효율

- 성능: 5개 예제 프로그램 기준 실행 1초 이내
- 효율성: 명령어 주소 인코딩 및 GUI 실행 흐름 최적화로 메모리 사용 효율화

#### 규격/표준

- 파일 형식 준수: .uco 파일의 코드 형식 준수
  - 1~10열: 레이블
  - 11열: 공백
  - 12열 이후: 명령어/피연산자
- 주석 규칙: %로 시작하는 행은 주석으로 인식

#### 환경

- 작동 환경: Ubuntu(운영체제)
- 개발 환경: C언어, GTK 3.0, GCC 컴파일러

#### 신뢰성 및 안정성

- 신뢰성: 어셈블 및 실행 오류 시 GUI/CLI 모두에서 로그 출력
- 안전성: 잘못된 입력에도 시스템 종료 없이 오류 메시지 출력 후 정상 종료

#### 유지보수성

- 모듈 구조: 어셈블러, 실행기, GUI 모듈을 사용하여 분리 구조로 관리

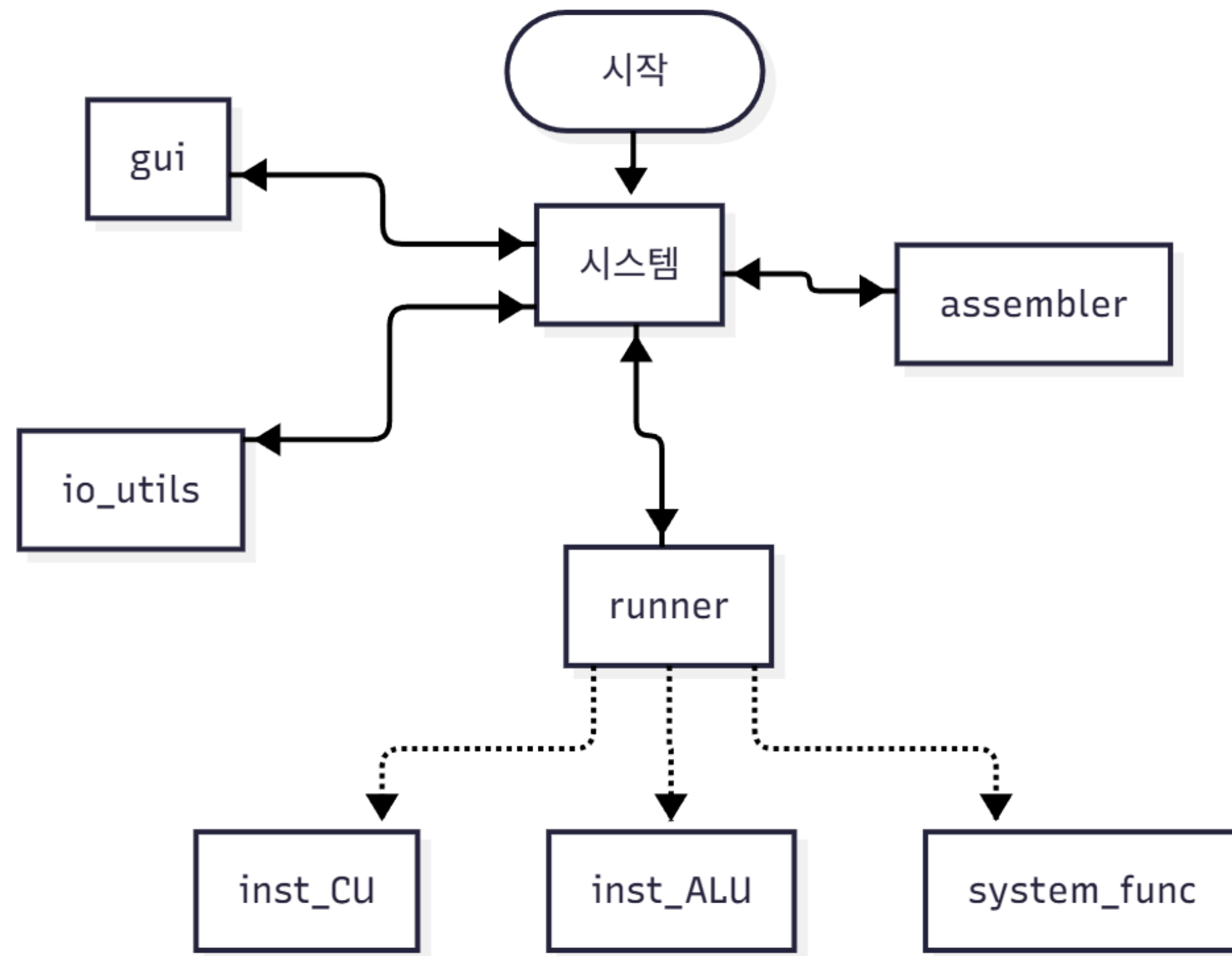
#### 데이터 및 명령어 규칙

- 데이터 타입 제한: 정수형만 지원
- 주소 지정 방식:
  - 연산 명령: 스택 기반(0주소)
  - 상수/메모리 접근/분기 명령: 피연산자 1개 사용(1주소)

PROGRAM & DATA DESIGN

# 프로그램 및 데이터 구조 설계

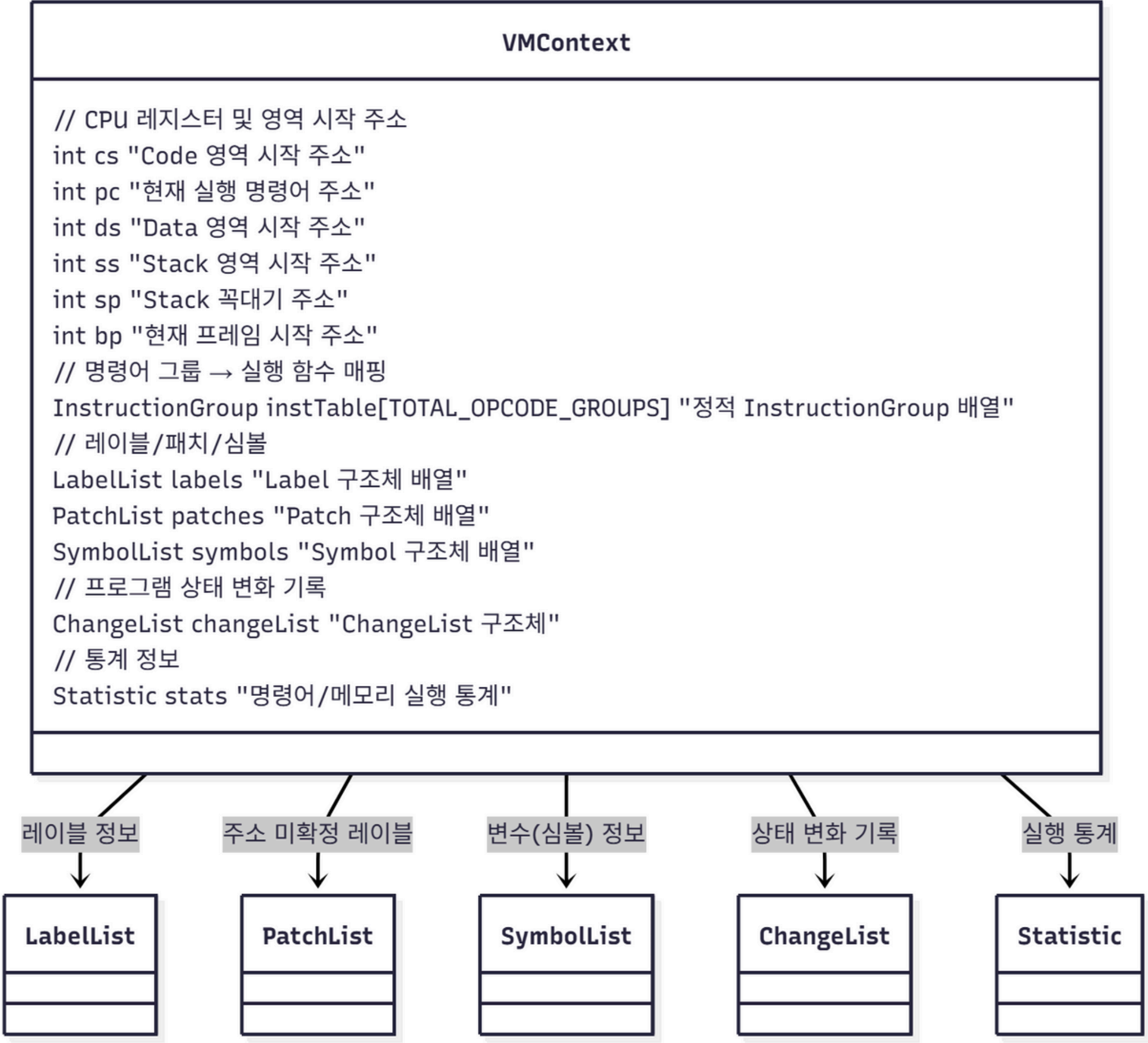
### 03 프로그램 및 데이터 구조 설계 - 프로그램 구조 설계



### 프로그램 구조

- 흐름 표시:
  - 실선 → 프로그램 실행 흐름
  - 점선 → 모듈 간 관계
- 프로그램 시작:
  - io\_utils 또는 GUI → .uco 파일 입력 → assembler 전달
- 어셈블 후 실행:
  - runner 호출 → inst\_CU, inst\_ALU, system\_func
  - 모듈 호출 → mU-Code 한 줄씩 실행
- 모듈 역할:
  - io\_utils: .uco 파일 입력, .lst 파일 출력
  - assembler: mU-Code 어셈블
  - runner: 어셈블 명령어 실행
  - inst\_ALU: 단항/이항 연산 명령어 처리
  - inst\_CU: 함수 호출, 데이터 이동, 분기 명령어 처리
  - system\_func: 시스템 함수 처리
  - GUI: GUI 생성 및 이벤트 처리

03 프로그램 및 데이터 구조 설계 - 데이터 구조 설계



VMContext 구조

- 가상 머신의 중앙 제어 구조체로 프로그램 실행 상태를 일관되게 관리함
- 레지스터로 CPU 상태를 저장함
- InstructionGroup 배열로 명령어 코드와 실행 함수를 매핑해 실행 효율 향상을 도모함
- LabelList / PatchList / SymbolList로 레이블, 미해결 주소, 심볼 정보 관리
- ChangeList로 실행 이력 추적, Statistic으로 실행 통계 수집 → 가상 머신의 전체 상태를 통합적으로 보관 및 제어하는 구조

### 03 프로그램 및 데이터 구조 설계 - 데이터 구조 설계

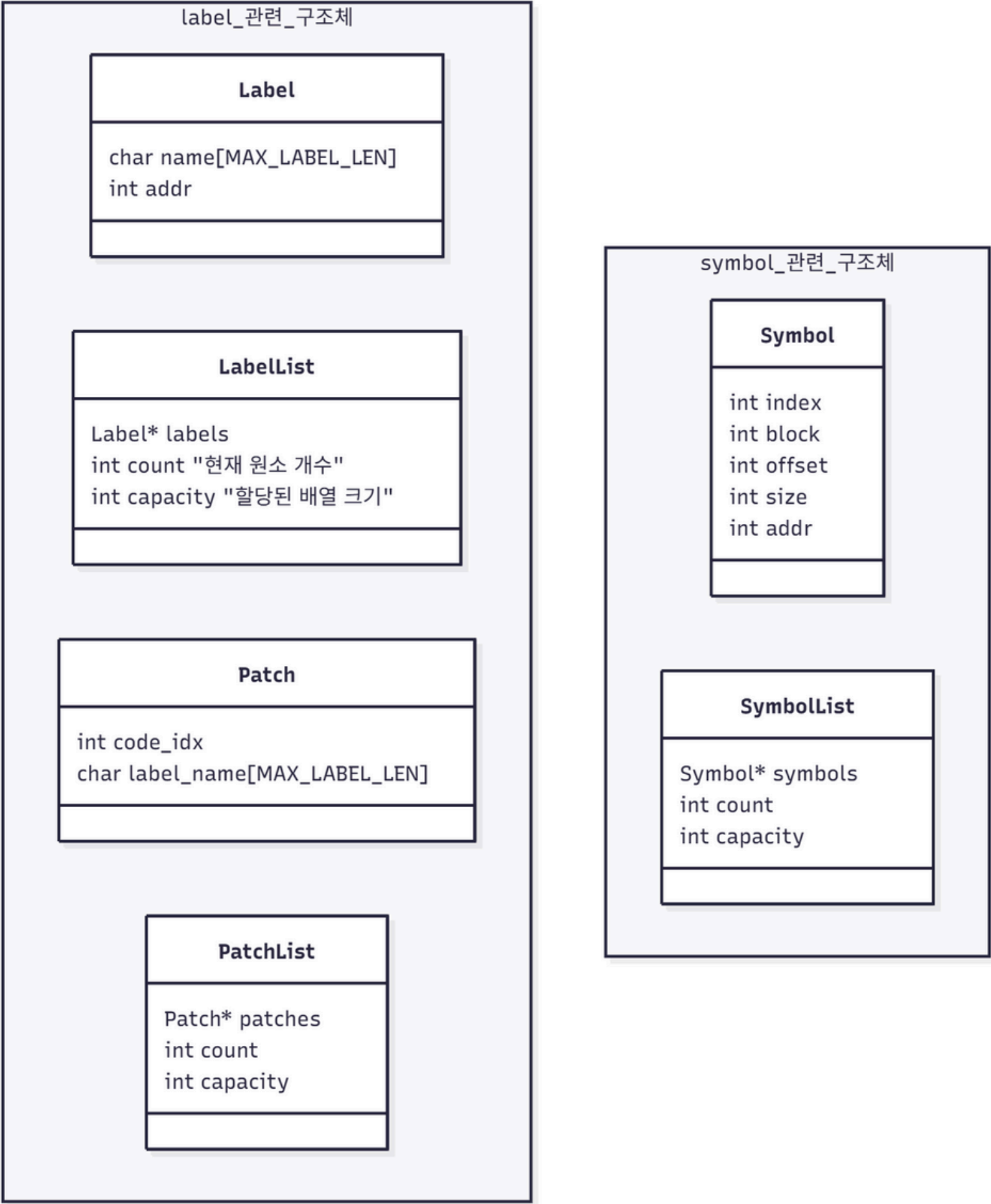
## 명령어 데이터 구조



- 명령어 저장: 32비트 int
  - 상위 6비트 → opcode
  - 하위 26비트 → 피연산자(operand), 음수 표현 가능 정수
    - 인자가 없는 경우 → 영향 x
    - 인자가 1개 있는 경우 → 레이블은 주소 변환 (시스템 함수의 경우 음수값)
    - 인자가 2개 있는 경우 → 변수는 변수 인덱스를 새로 부여(1주소 제한 지킴)
- Opcode 구조:
  - 상위 3비트 → 명령어 그룹
  - 하위 3비트 → 그룹 내 인덱스  
→ 기능 단위(예: 함수 정의/호출, 흐름 제어)로 분류
- InstructionGroup 구조체:
  - opcode별 명령어 처리 함수 포인터 보유
  - 배열로 관리 → opcode에 맞는 함수 호출 가능
- 장점:
  - 비트 연산만으로 함수 포인터 조회 가능 → 실행 속도 향상
  - enum으로 opcode 정의 → 코드 가독성/유지보수성 개선



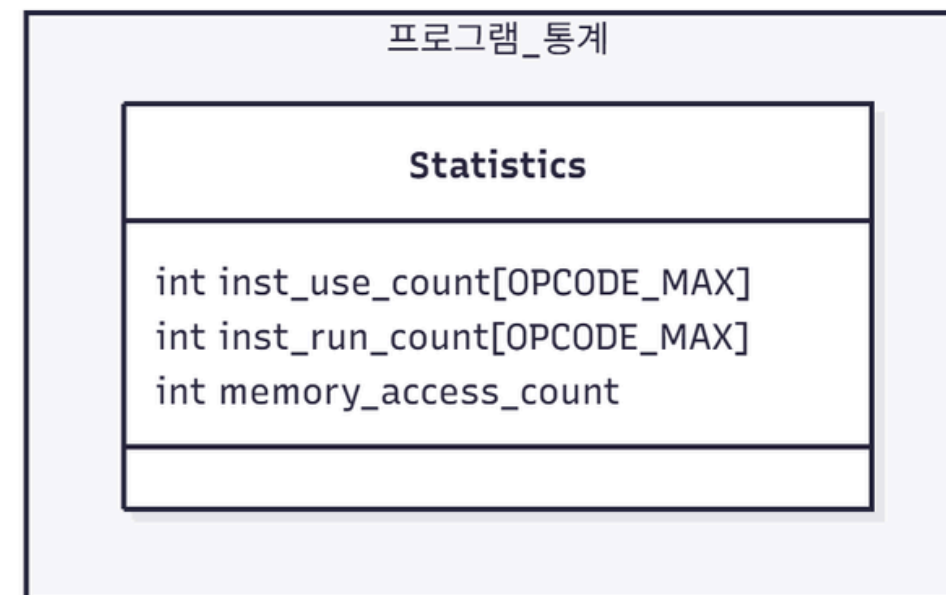
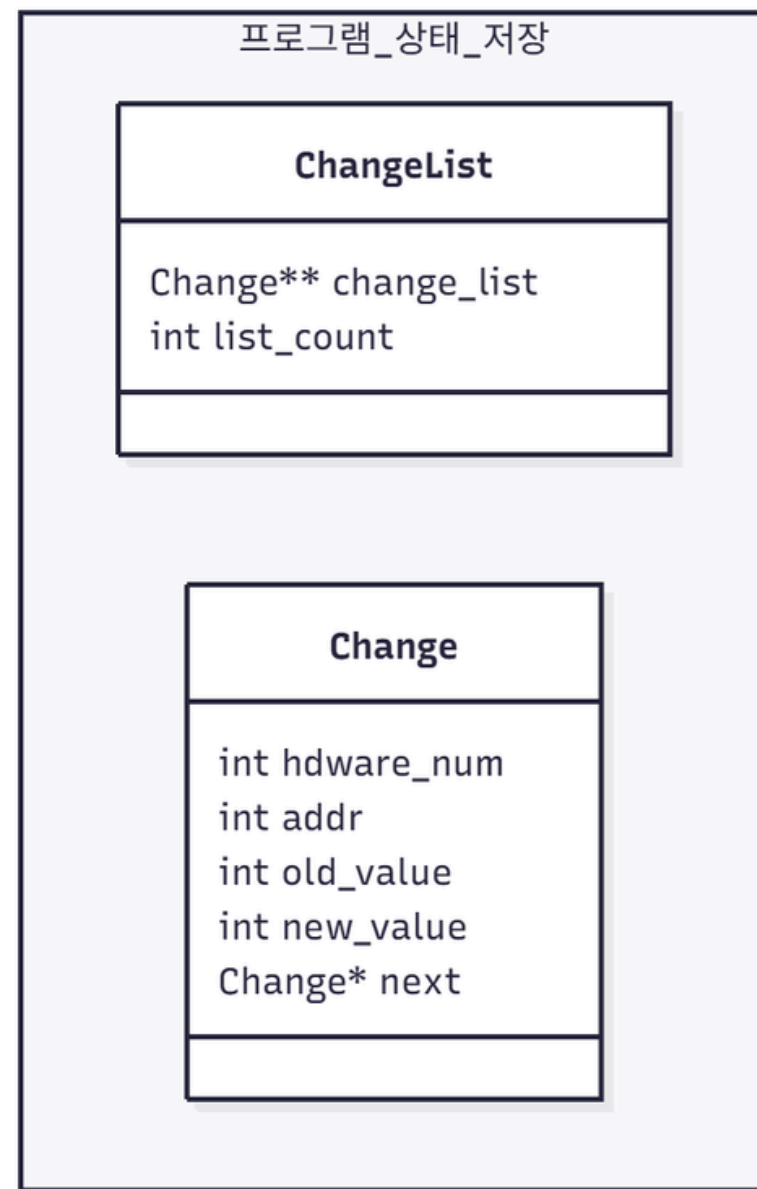
03 프로그램 및 데이터 구조 설계 - 데이터 구조 설계



어셈블 관련 데이터 구조

- Label: 레이블 이름과 코드 내 주소 저장. 시스템 함수와 사용자 정의 레이블 관리
- Patch: 주소가 확정되지 않은 코드 인덱스와 참조 레이블 기록. 어셈블 종료 시 LabelList를 참고해 주소 갱신
- Symbol: 변수 인덱스, 블록, 오프셋, 크기 정보 저장. 지역 변수는 BP 갱신 시 주소 재계산 필요
- 관리 구조: LabelList, PatchList, SymbolList로 동적 배열 관리  
→ 실행 중 레이블·미확정 레이블·심볼 정보를 유연하게 처리 가능

### 03 프로그램 및 데이터 구조 설계 - 데이터 구조 설계



### 실행 관련 데이터 구조

- Change: 하나의 상태 변경 기록
  - 대상 하드웨어, 주소, 이전 값/새 값, 다음 Change 포인터
- ChangeList: 각 사이클의 Change 연결 리스트를 관리
  - Change\*\* change\_list → 각 사이클 시작 포인터 배열
  - list\_count → 배열 크기 기록
- Statistic: .lst 파일 생성을 위한 통계 자료
  - 명령어 사용/실행 횟수, 메모리 접근 횟수 등 기록

## 03 프로그램 및 데이터 구조 설계 - 인터페이스 설계

### 인터페이스 설계

#### 데이터 흐름

- 입력 단계
  - .uco 파일 → io\_utils 모듈에서 읽어 Assembler로 전달
- 어셈블 단계
  - 명령어 분석 → Code 영역 저장
  - 레이블·변수 테이블, 명령어 통계 생성
- 실행 단계
  - Runner가 전역 배열 기반으로 명령어 실행
  - VMContext로 상태(스택, 메모리, 레지스터) 전달
- 검사 및 기록
  - 오류 검출 시 플래그 갱신
  - ChangeList 구조체에 실행 이력 저장
- 출력 단계
  - 실행 결과 → .lst 파일 저장
  - GUI에 스택/메모리 변화 표시

#### 주요 함수 시그니처

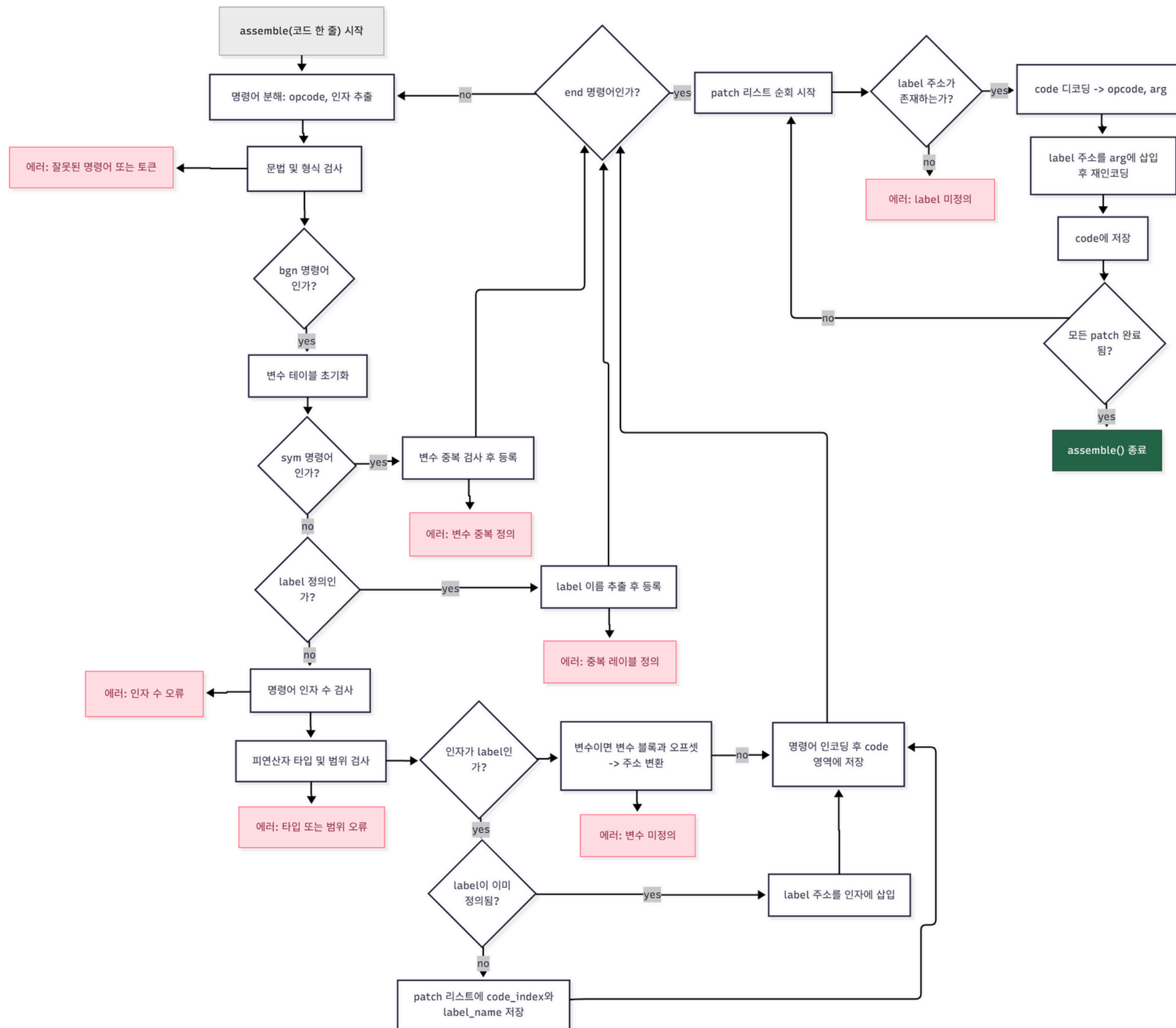
- int loadUco(const char\* path, char\*\*\* lines, int\* count)  
— 파일 로드
- int saveLst(const char\* path) — 파일 저장
- int assemble(char\*\* lines, int line\_count) — 어셈블 수행
- int run() / step() — 코드 실행
- void (\*execInst)(int arg) — 명령어 수행할 각 함수 포인터
- void startGui() — GUI 실행



ALGORITHM & UI DESIGN

# 알고리즘 및 UI 설계

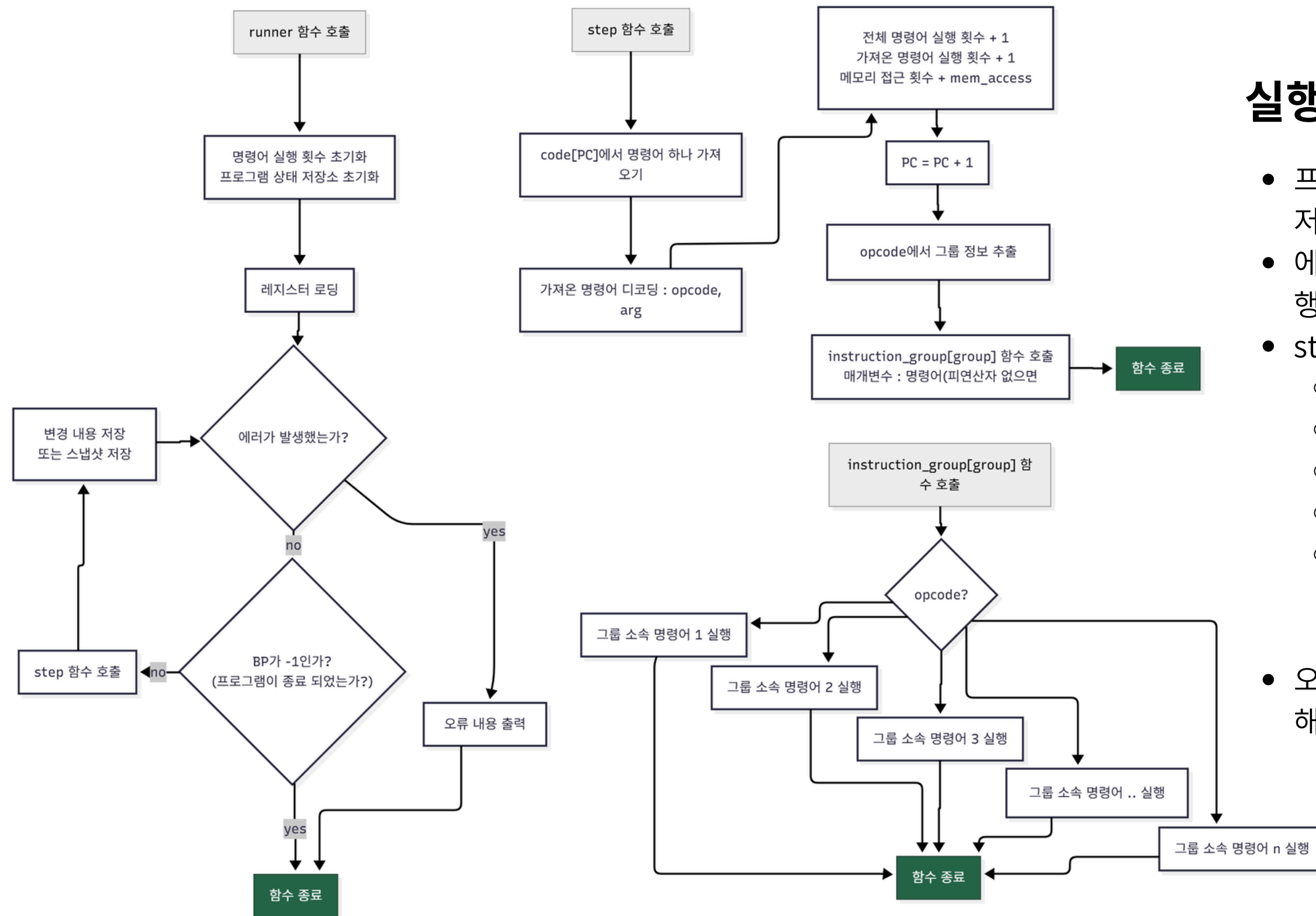
## 04 알고리즘 및 UI 설계 - 알고리즘 설계



## 어셈블 알고리즘

- assemble(line) 호출: .uco 파일에서 한 줄씩 읽은 코드를 assemble 함수에 전달하여 처리
- 구문 분석: 명령어(opcode)와 인자를 추출하고 문법과 형식 검사 수행
- 변수/심볼 처리:
  - bgn 명령어 → 변수 테이블 초기화
  - sym 명령어 → 변수 정의 및 중복 검사
- 레이블 처리:
  - label 정의 → 레이블 등록
  - 인자가 레이블이면 정의 여부 확인, 미정의 시 patch 리스트에 저장
- 명령어 인코딩: 검증이 완료된 명령어를 코드 영역에 저장
- Patch 처리: assemble 종료 후 patch 리스트 순회 → 실제 레이블 주소 삽입
- 오류 처리: 잘못된 명령어, 변수 중복/미정의, 레이블 중복/미정의, 인자 오류 등 모두 로그 출력

## 04 알고리즘 및 UI 설계 - 알고리즘 설계

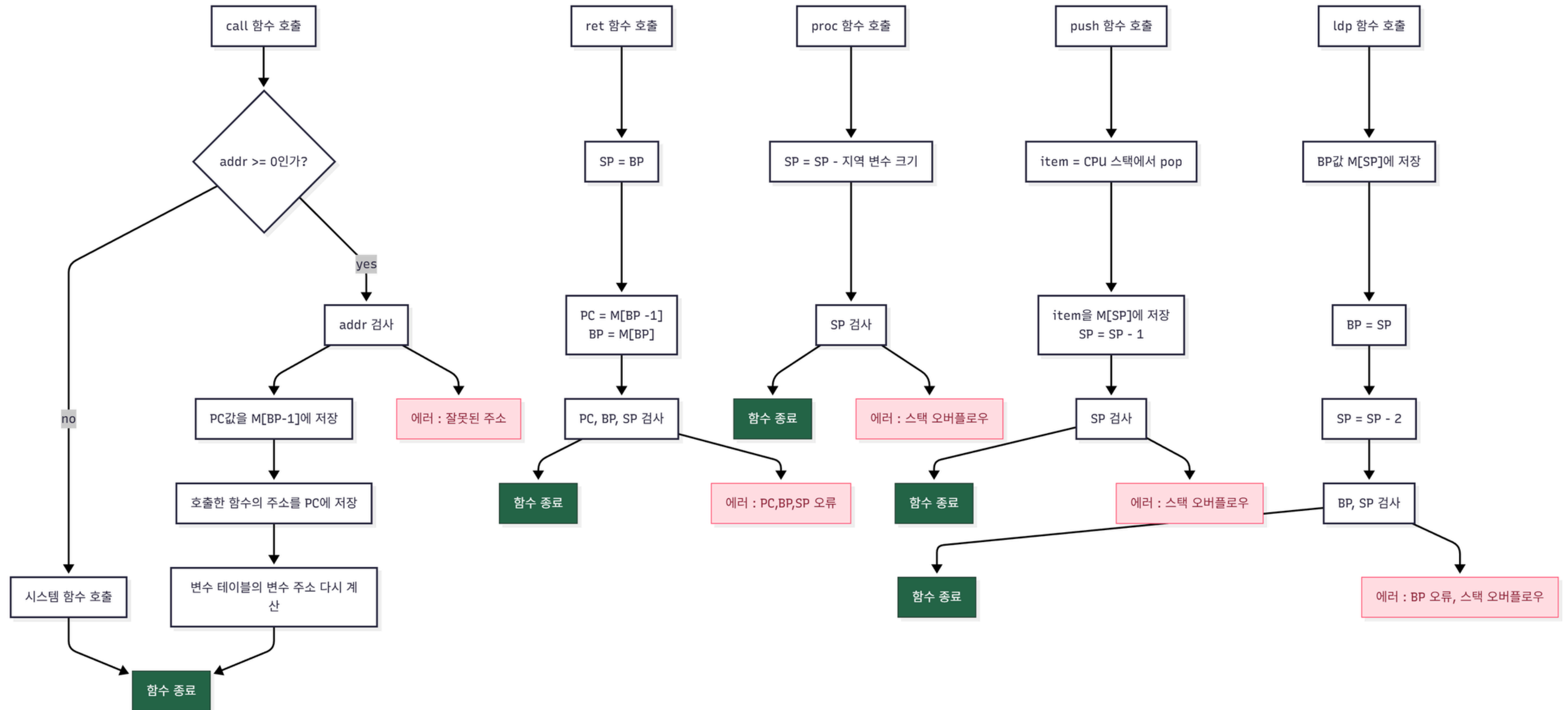


## 실행 알고리즘

- 프로그램 실행 및 분석 준비 : 통계 정보 및 프로그램 상태 저장소 초기화, 레지스터 값 로드
- 에러 발생 여부, 프로그램 종료 여부 확인하며 명령어 실행 변경 내용 저장 반복
- step() 함수 호출 : 명령어 한줄씩 처리
  - 메모리에서 명령어 인출
  - 명령어 디코딩 : opcode, arg
  - 실행 횟수, 메모리 접근 횟수 갱신
  - 해당 명령어 분석
  - 매핑된 명령어 그룹 함수 호출
    - 명령어 처리함수 호출 (\*execInst)(int arg) : opcode에 맞는 명령어 처리 함수 호출
- 오류 발생시 : 에러 레지스터 세트 및 상태 레지스터 참고해 오류 내용 출력

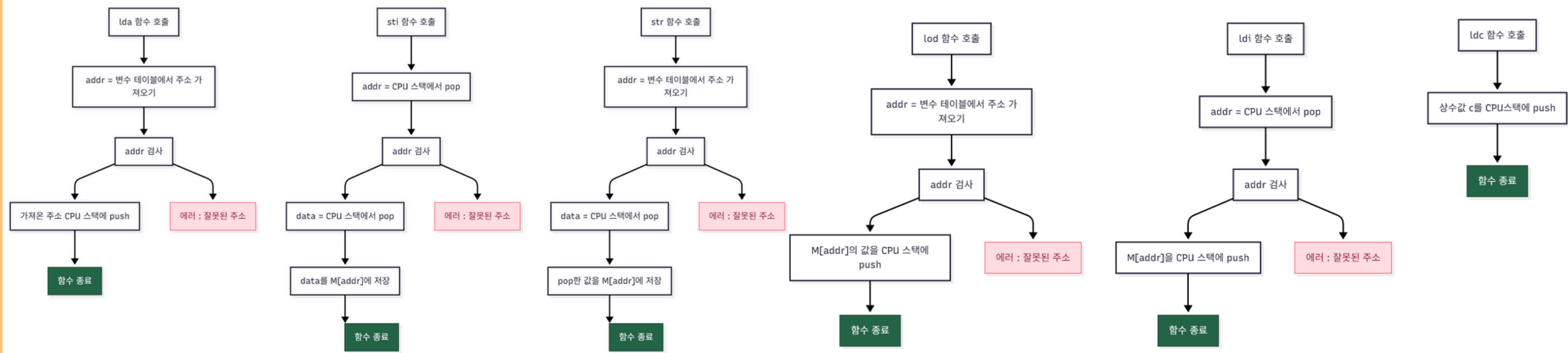
## 04 알고리즘 및 UI 설계 - 알고리즘 설계

### 함수 정의 및 호출 명령어



# 04 알고리즘 및 UI 설계 - 알고리즘 설계

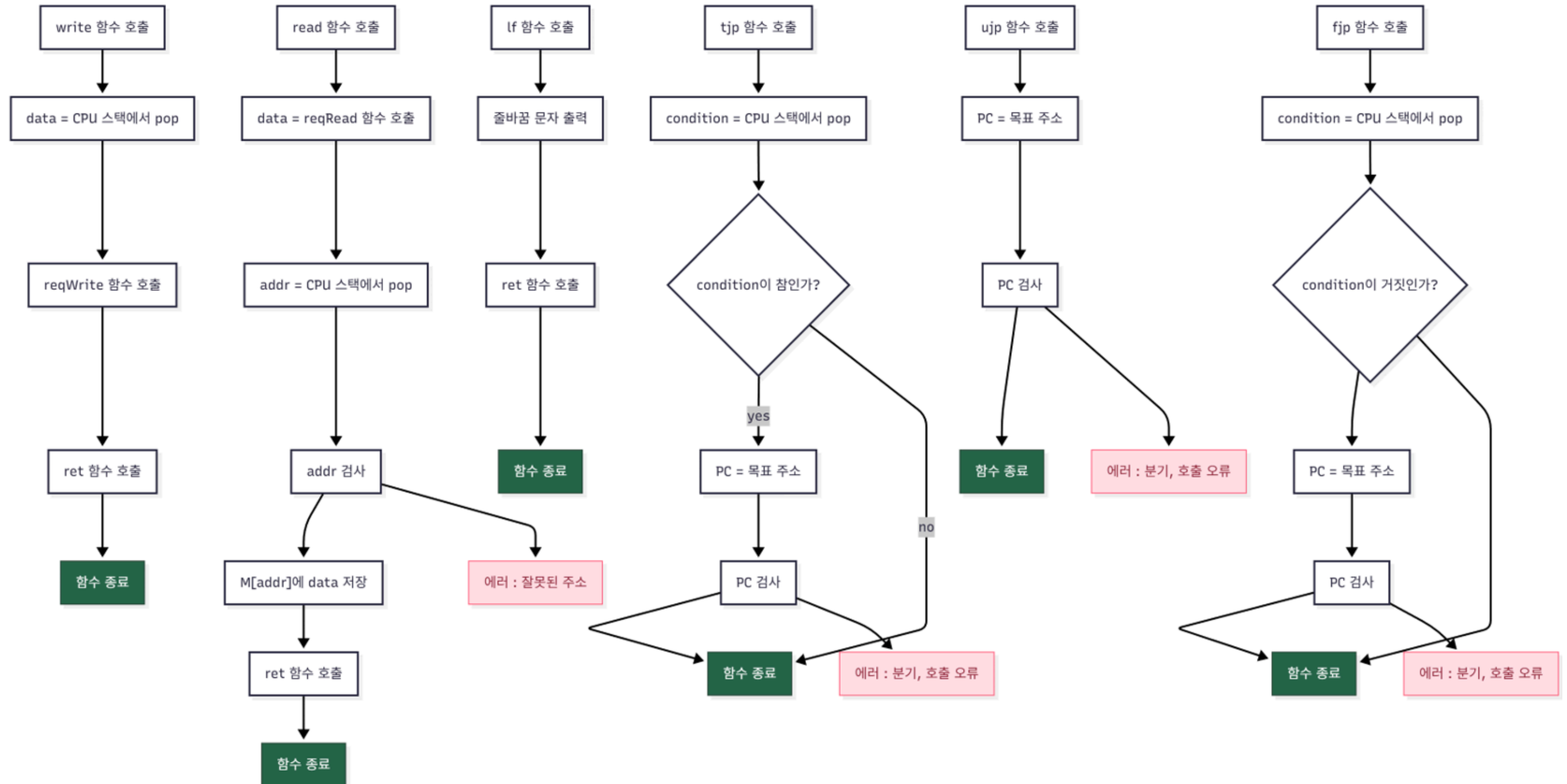
## 데이터 이동 명령어





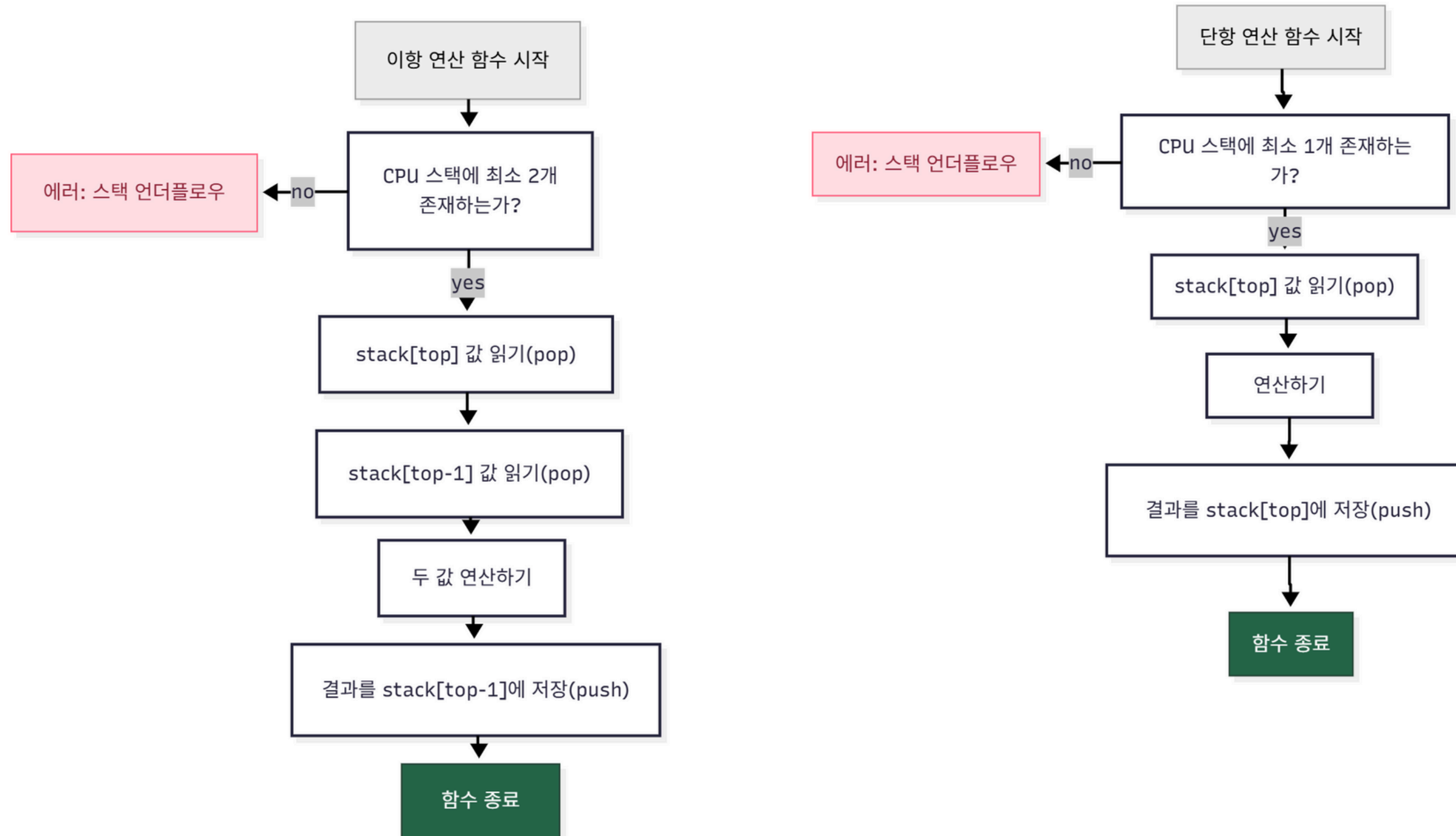
## 04 알고리즘 및 UI 설계 - 알고리즘 설계

### 시스템 함수 및 분기 명령어

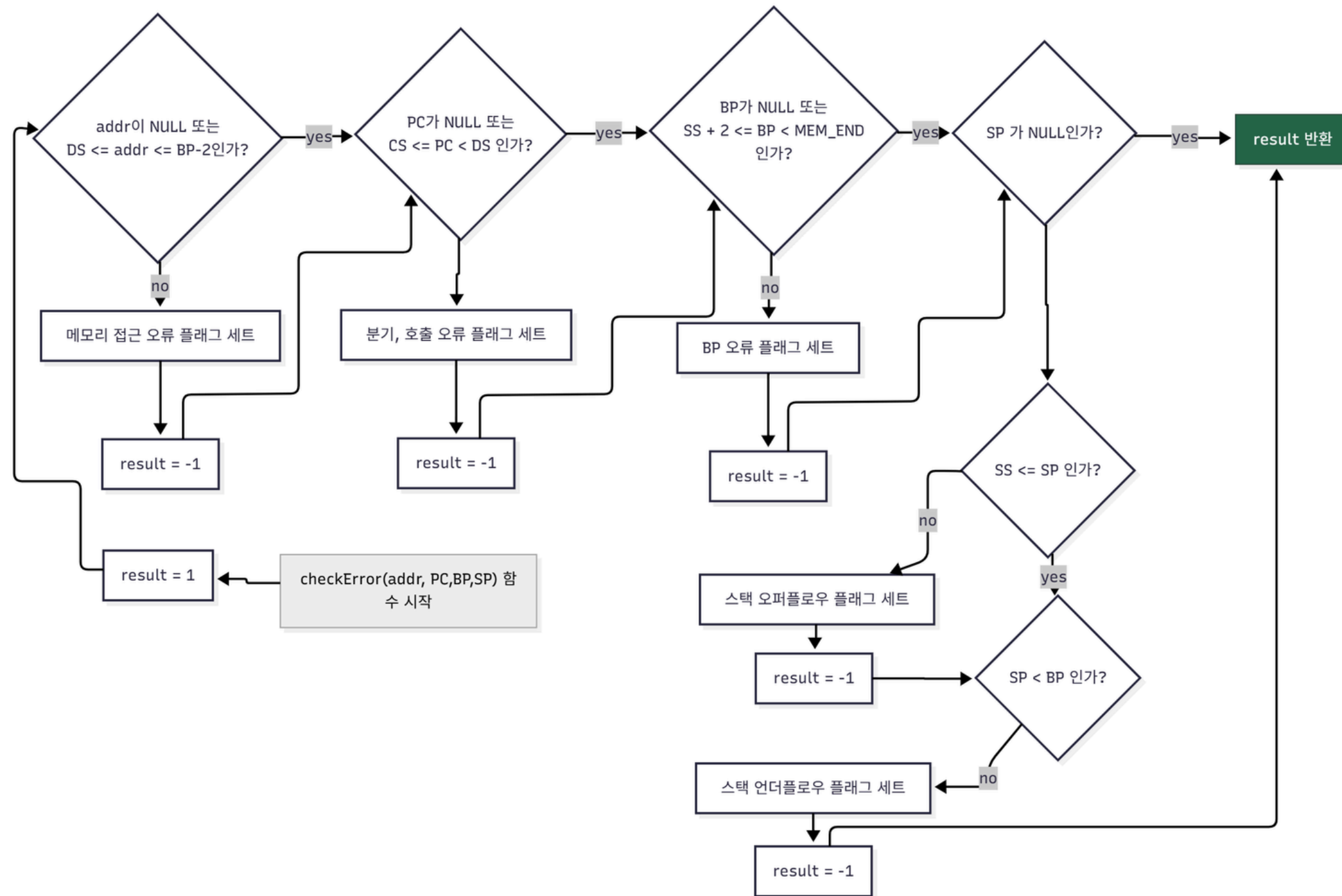


## 04 알고리즘 및 UI 설계 - 알고리즘 설계

### 이항 연산 및 단항 연산 명령어



## 04 알고리즘 및 UI 설계 - 알고리즘 설계

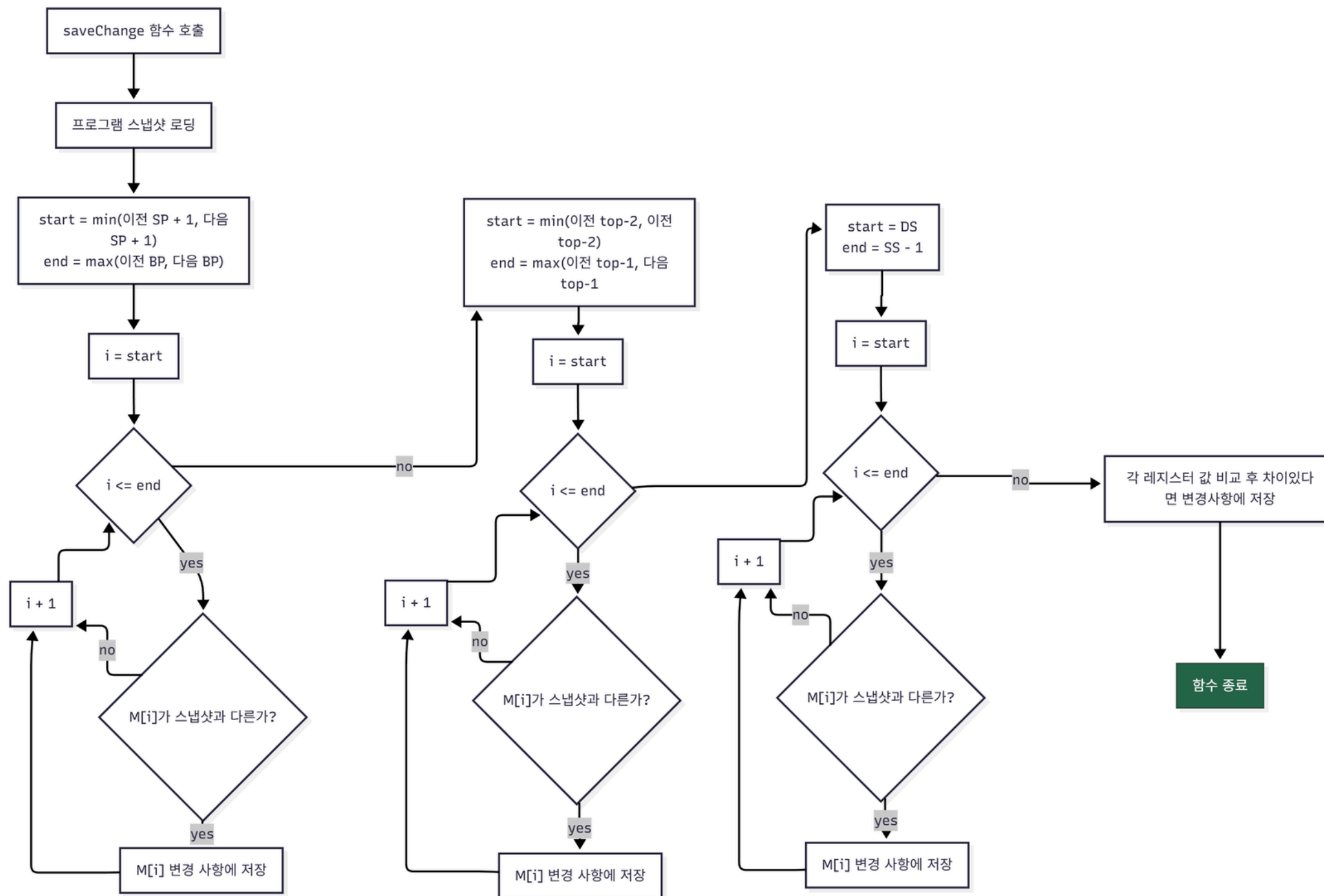


## 오류 검사 알고리즘

- checkError(addr, PC, BP, SP) 호출 : 검사를 원하는 변수는 해당 값, 원하지 않으면 NULL 전달
- 접근하려는 주소 범위 검사
- 분기하려는 PC값 검사
- 변경하려는 BP값 검사
- 변경하려는 SP값 검사
- 만약 정상 범위 내에 없다면, 에러 및 관련 플래그 세트



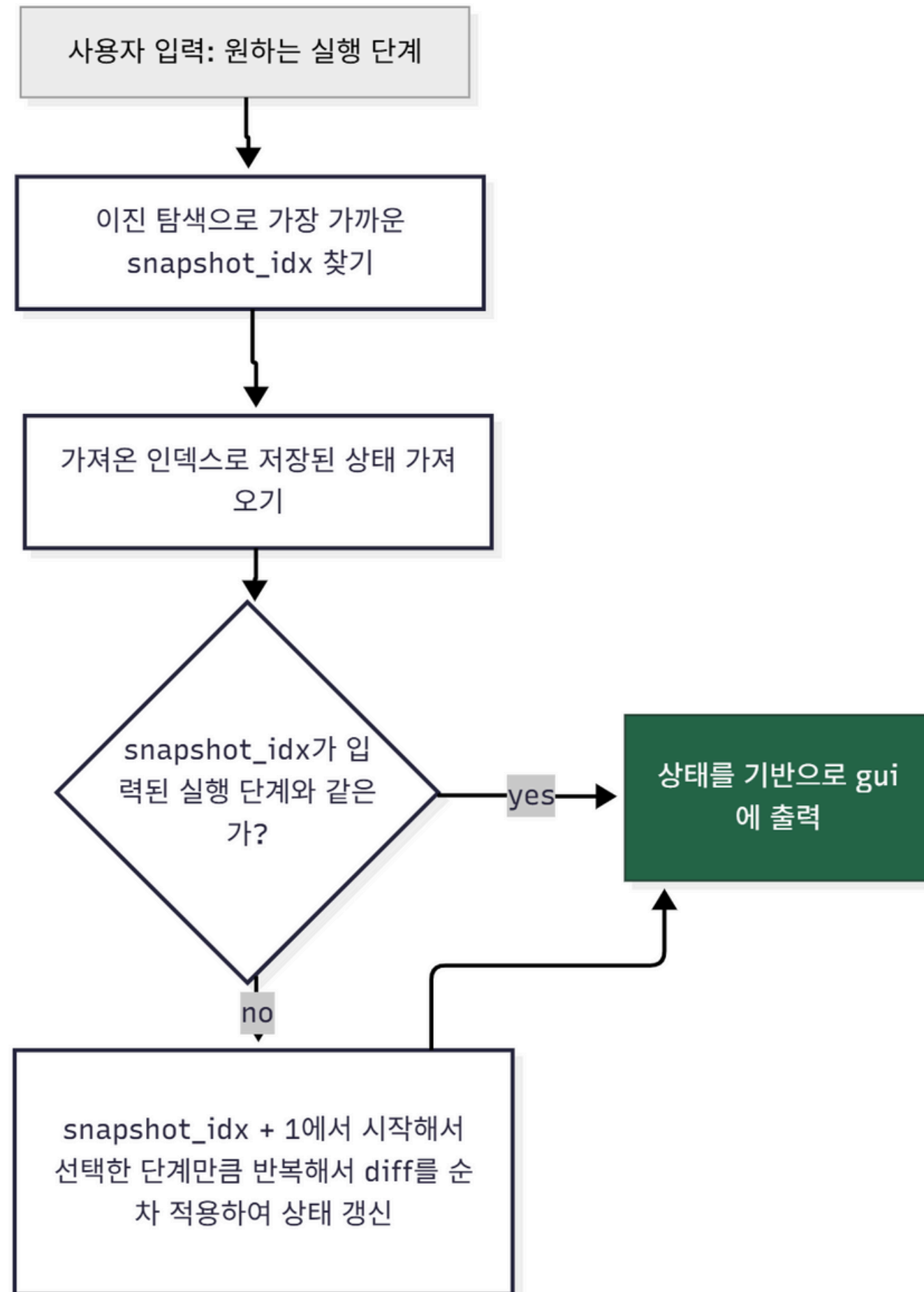
## 알고리즘 및 UI 설계 - 알고리즘 설계



## 변경 내용 저장 알고리즘

- 이전 단계의 스냅샷 로딩
- Stack 영역 검사
- CPU 스택 검사
- Data 영역 검사
- 각 레지스터 검사
- 만약 변경 사항을 발견하면 Change 구조체 연결리스트를 만들어 ChangeList 구조체에 저장한다.

## 04 알고리즘 및 UI 설계 - 알고리즘 설계



## GUI 실행 흐름 제어

- 상태 저장 방식
  - 실행기는 일정 주기마다 전체 상태를 스냅샷(snapshot)으로 저장
  - 나머지 단계는 변경된 부분만 기록
- 단계 이동 알고리즘
  - 사용자가 단계 선택 → 탐색으로 가장 가까운 스냅샷 탐색
  - 선택 단계 = 스냅샷 → 그대로 출력
  - 선택 단계 ≠ 스냅샷 → 스냅샷 이후 변경점 순차 적용 후 GUI 갱신
- 장점
  - 메모리 절약 (전체 저장 대신 간격 스냅샷)
  - 빠른 복원 (탐색 + 변경점 적용)
  - 긴 실행 기록에도 확장성과 GUI 응답성 확보

## 04 알고리즘 및 UI 설계 - UI 설계

### CLI 설계

#### 정상 시나리오

```
>>> uci ucode.uco  
어셈블 시작  
어셈블 완료  
실행시작  
HelloWorld  
실행 완료  
./Statistics.lst 경로에 통계 파일이 생성되었습니다.  
>>>
```

#### 파일 경로 잘못됨

```
>>> uci ucode.ico  
“unicode.ico” 파일을 찾을 수 없습니다.  
>>>
```

## 04 알고리즘 및 UI 설계 - UI 설계

### 어셈블 단계 오류

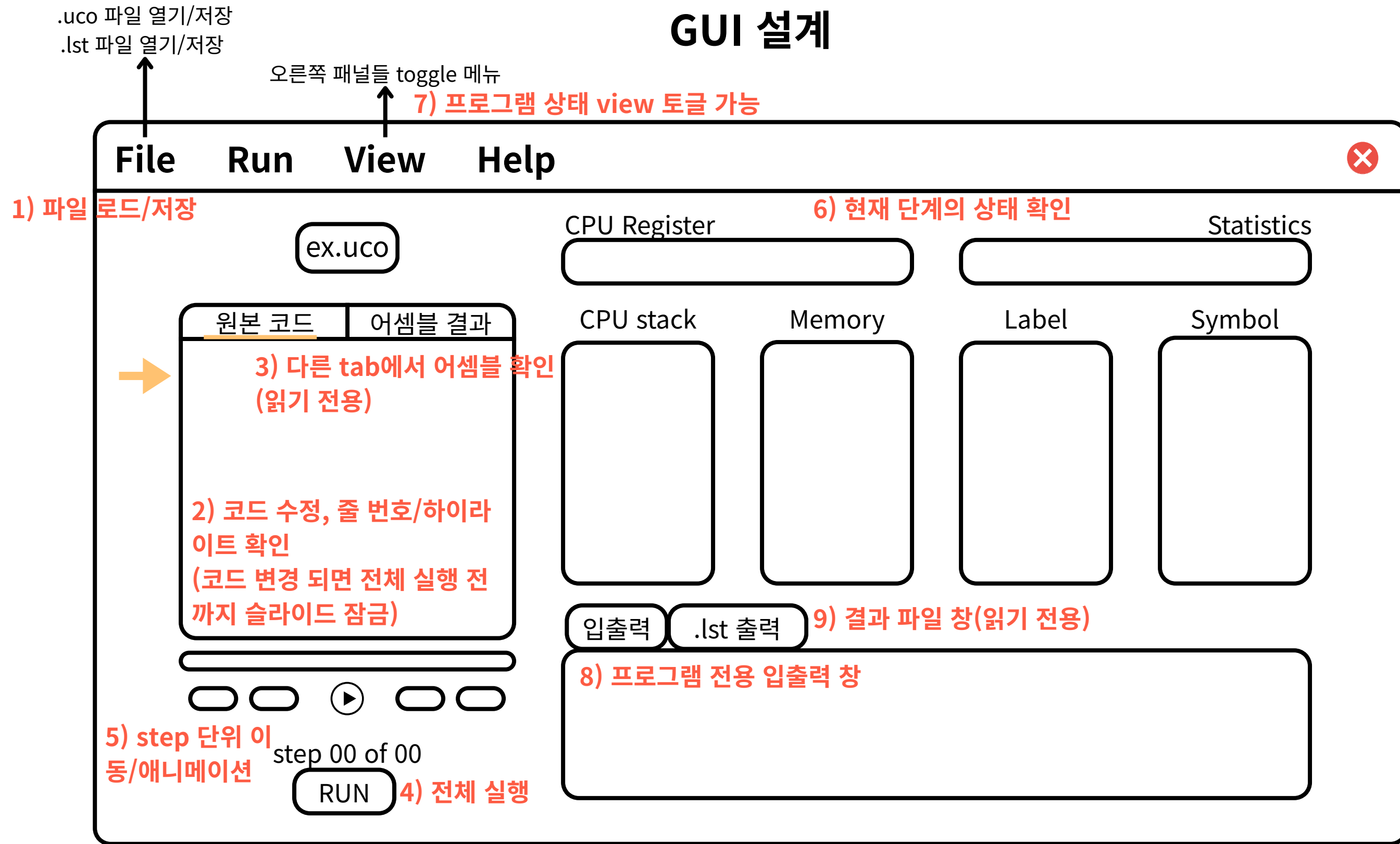
```
>>> uci ucode.uco
어셈블 시작
16행 : kall sub1
error: 존재하지 않는 명령어 입니다.
>>>
```

### CLI 설계

### 실행 단계 오류

```
>>> uci ucode.uco
어셈블 시작
어셈블 완료
실행시작
error : stack overflow
실행 종료됨
./Statistics.lst 경로에 통계 파일이 생성되었습니다.
>>>
```

04 알고리즘 및 UI 설계 - UI 설계





IMPLEMENTATION STRATEGY

추진 전략

## 05 추진 전략

### 개발 방법론

폭포수 모델(Waterfall Model)

문제 정의 → 요구분석 → 설계 → 구현 → 테스트 → 완료

### 정보 수집 방법

- 강의 자료
- 브라우저 검색
- AI
- GTK 라이브러리 공식 문서

### 개발 도구

- C, linux
- GTK3 라이브러리
- Github

### 역할 분담

#### 공통

입출력 기능

테스트 및 피드백

결과 보고서

#### 이강석

명령어 함수

실행기

테스트 코드

CLI

#### 권기영

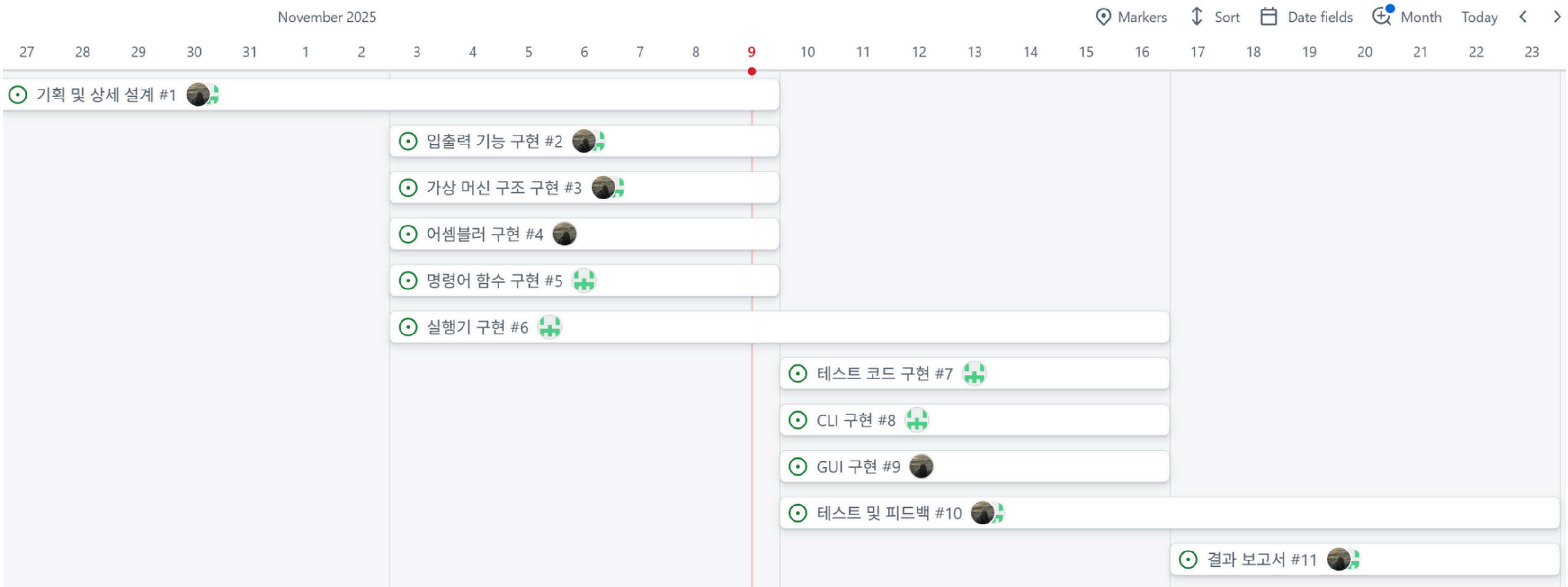
가상 머신 구조

어셈블러

GUI

05 추진 전략

개발 기간: 2025년 10월 27일 ~ 2025년 12월 03일





팀 UCI

컴퓨터공학과 20220744 이강석(팀장)

컴퓨터공학과 20240058 권기영